Week 1 Lecture Notes: Python and Environment Review

From a raw flight log to engineering answers

AI for Aerospace & Mechanical Engineering (PhD), Block 0

> **Provenance:** new content. The instructor's "environment setup + ML intro" PDF was not available when these notes were written; reconcile with it when it is. Python tooling is rewritten (not copied) from W3Schools (NumPy intro, Pandas Read CSV, Pandas Analyzing DataFrames, Matplotlib Pyplot, Git Introduction) and GeeksforGeeks (Python Virtual Environment, Anaconda virtual environments, Getting started with Jupyter, DataFrame describe()). Turning-flight relations are adapted from J. D. Anderson, *Introduction to Flight*, Ch. 6.

1. The problem

A flight-test engineer gives you a CSV file: five minutes of a small fixed-wing UAV test flight. A 3-axis accelerometer and a 3-axis gyroscope are logged at 50 Hz together with barometric altitude, pitot airspeed and outside air temperature. The chief engineer asks:

1. **Q1.** What was the sampling rate, and did the logger lose data?
2. **Q2.** What was the peak load factor? Was the 2.0 g structural limit exceeded?
3. **Q3.** How noisy are the gyroscopes while the aircraft is parked?
4. **Q4.** What was the peak climb rate?

These are not machine-learning questions yet, but every model later in the course will inherit the answers. A model trained on data with an unknown time step, unflagged gaps or uncleaned spikes learns the defects. Week 1 therefore builds the habits and tools for looking at data honestly: a reproducible environment, NumPy, Pandas and Matplotlib.

2. A reproducible environment

A result that runs only on the author's laptop is not a result. An *isolated environment* is a private Python installation with its own package versions, so that two projects (or two students) never disturb each other. Two tools create one:

- **conda** (`conda create -n aero-ai python=3.11`, `conda activate aero-ai`) manages Python itself and compiled libraries; we will need it for PyTorch from Week 9.
- **venv** (`python -m venv .venv`, then `pip install -r requirements.txt`) is built into Python and is lighter.

The *recipe* of the environment is a text file (`environment.yml` or `requirements.txt`) with pinned versions; it lives in Git next to the code. **Jupyter** notebooks mix code, results and narrative, which suits labs; anything reused goes into `.py` modules that notebooks import. **Git** records every version of the work: `git switch -c` creates a branch, `git add` stages changes, `git commit` records them

and `git push` publishes them. Running `git status` before each commit is the cheapest bug-prevention habit in this course.

3. NumPy: arrays, axes and vectorisation

A Python list holds references to separate objects; a NumPy `ndarray` is one contiguous block of numbers of a single type. Whole-array operations therefore run in compiled code. In the lab, the norm of 200 000 acceleration vectors takes about ten times less time with `np.sqrt(np.sum(a**2, axis=1))` than with a list comprehension.

For a log stored as an array of shape $(N, 3)$, axis 0 runs over time samples and axis 1 over the x, y, z components. Reducing over `axis=0` gives one value per component (for example the bias of each gyro); reducing over `axis=1` gives one value per sample (for example the norm). **Broadcasting** lets shapes $(N, 3)$ and $(3,)$ combine: the vector is applied to every row, so `a - a.mean(axis=0)` removes the per-axis bias without a loop.

Worked Example 1: which standard deviation?

Five gyro samples at rest are $x = [0.2,\ -0.4, 0.1, 0.5,\ -0.4]$ deg/s. The mean is $\bar{x} = 0$ and the squared deviations sum to $0.04 + 0.16 + 0.01 + 0.25 + 0.16 = 0.62$.

$$\sigma = \sqrt{\tfrac{1}{N}\sum (x_i - \bar{x})^2} = \sqrt{0.62/5} = 0.352, \qquad s = \sqrt{\tfrac{1}{N-1}\sum (x_i - \bar{x})^2} = \sqrt{0.62/4} = 0.394.$$

`np.std(x)` returns σ (`ddof=0`) but `pd.Series(x).std()` returns s (`ddof=1`): the same data give results 12 % apart. For a noise specification, s is the right estimate (an unbiased variance from a sample); write `ddof=1` explicitly in both libraries.

4. Load factor from an accelerometer

An accelerometer measures **specific force f**: the non-gravitational force per unit mass. Parked on the ground it reads $+g$ upward. The load factor is

$$n = \frac{\| \mathbf{f} \|}{g_0}, \qquad g_0 = 9.80665 \text{ m/s}^2 .$$

In a level coordinated turn with bank angle φ, the vertical component of lift balances the weight, $L \cos\varphi = W$, so

$$n = \frac{L}{W} = \frac{1}{\cos\varphi}.$$

A 30° bank gives $n = 1.155$, and the 2.0 g limit corresponds to a 60° bank.

Worked Example 2: one sample by hand

A sample reads $\mathbf{f} = (1.2,\ -0.4, 19.6)$ m/s². Then $\| \mathbf{f} \|^2 = 1.44 + 0.16 + 384.16 = 385.76$, $\| \mathbf{f} \| = 19.641$ m/s² and $n = 19.641/9.80665 = 2.003$. One sample is above the limit; whether the *airframe* exceeded it is a separate question (Section 6).

5. Pandas: reading a dataset like an engineer

`pd.read_csv` returns a `DataFrame`, a table of named NumPy columns. Three calls give a first impression: `head()` (does the file look right?), `info()` (types and non-null counts) and `describe()` (count, mean, standard deviation, min, quartiles and max per column). In our log, `describe()` already shows two problems: `baro_alt_m` and `airspeed_mps` have 75 fewer values than the other columns (missing data), and `accel_z_mps2` has a maximum of 50.3 m/s², about 5 g.

6. Answering the four questions

Q1, sampling and gaps. The sample rate is $f_s = 1/\operatorname{median}(\Delta t)$. The median matters: in the excerpt $t = [99.94, 99.96, 99.98, 101.20, 101.22]$ s, the differences are $0.02, 0.02, 1.22, 0.02$ s. Their mean (0.32 s) is ruined by one gap, while the median (0.02 s) gives the correct 50 Hz. The Nyquist frequency is 25 Hz. Flagging any step longer than 1.5 nominal steps finds one gap of $1.22/0.02 - 1 = 60$ missing samples, and `df.isna().mean()` shows 0.5 % NaN in the barometer and airspeed channels.

Q2, spikes versus manoeuvres. The raw peak is 5.13 g at $t = 66.68$ s, but its neighbours are all 1.06 g: a one-sample (20 ms) event is a sensor glitch, because a real manoeuvre builds up over seconds. A centred **rolling median** removes such spikes while preserving manoeuvres.

Worked Example 3. For the window $[1.00, 1.01, 5.10, 0.99, 1.00]$, the mean is $9.10/5 = 1.82$ g (the spike leaks in) while the median of the sorted window $[0.99, 1.00, 1.00, 1.01, 5.10]$ is 1.00 g.

After a 5-sample rolling median the true peak is **1.80 g** during the pull-up at $t \approx 192$ s: the 2.0 g limit was not exceeded.

Q3, noise at rest. Selecting $t < 25$ s with a boolean mask gives three gyro channels with a bias below 0.02 deg/s and a sample standard deviation of about **0.30 deg/s**, with near-Gaussian histograms (white noise). This number feeds the navigation team's Kalman filter.

Q4, climb rate. Differentiation amplifies noise.

Worked Example 4. With $h = [250.0, 250.3, 250.5]$ m at $t = [80.00, 80.02, 80.04]$ s, the central difference is

$$\dot{h}(80.02) \approx \frac{h(80.04) - h(80.00)}{0.04} = 12.5 \text{ m/s}.$$

If each sample carries barometric noise $\sigma_h = 0.8$ m, the numerator has standard deviation $\sqrt{2}\,\sigma_h = 1.13$ m, so the estimate carries about ± 28 m/s of noise. We therefore resample the altitude onto a uniform 50 Hz grid (filling NaN and the gap by linear interpolation), smooth it with a 2 s centred window, and only then differentiate. The peak climb rate is about **16 m/s** at $t \approx 84$ s, consistent with the 15 m/s design value.

7. Matplotlib and the step to classes

`fig, axes = plt.subplots(4, 1, sharex=True)` produces aligned panels on one time axis: altitude, airspeed, load factor with a dashed 2.0 g line, and climb rate, with the logger gap shaded by `axvspan`. Every axis carries a unit, and every panel answers one question.

The lab ends by repeating all four answers with the `FlightLog` class in `src/aerospace_ai/utils.py`, which packages the same logic with a `@classmethod` constructor

(`from_csv`), computed `@property` attributes (`sample_rate_hz`), `__slots__`, `__repr__`, type hints and docstrings. Week 2 builds on this with a `Sensor` class hierarchy.

What we now know how to do

- Create, activate and pin an environment; branch, commit and push with Git.
- Replace loops with vectorised NumPy and choose the right `axis`; use broadcasting.
- Load a sensor CSV and read `head`, `info` and `describe` critically.
- Recover the sample rate robustly; find gaps and missing values.
- Separate sensor spikes from real manoeuvres with a rolling median.
- Estimate noise (stating `ddof`) and derivatives (smooth first) from noisy measurements.

Practice: in-class problems `in_class_problems/week01_problems.ipynb`, lab `notebooks/week01/week01_python_env.ipynb`, and Assignment 0 (`assignments/assignment00/`).